Data Mining Assignment

Lab 2

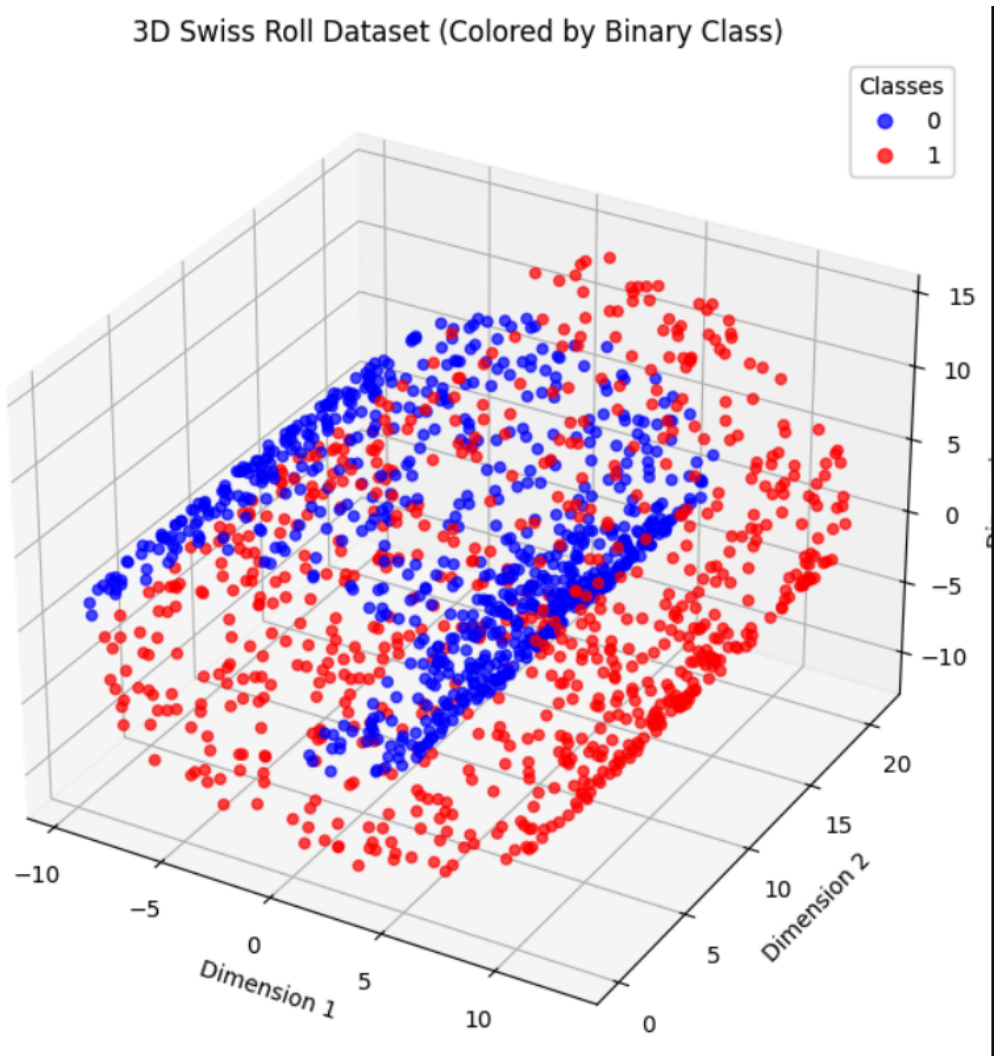
Done by:

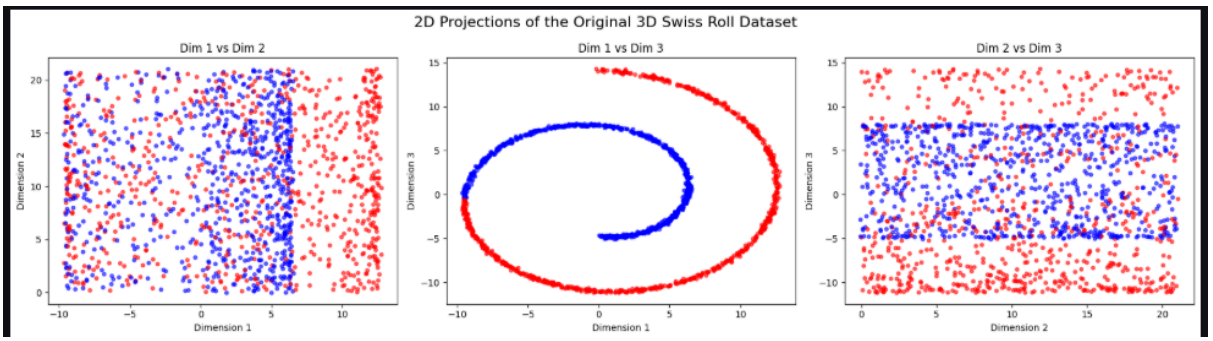
Shadi Reda Elattar - 8180
Omar Abourida - 8192

1. Dataset Preparation & Exploratory Data Analysis (EDA)

The Swiss Roll dataset (`n_samples=1500, noise=0.1`) was generated with binary labels splitting the roll along its median continuous position.

Challenges of Classifying the Swiss Roll: The primary challenge of the Swiss Roll dataset is its complex, non-linear manifold structure. In standard 3D Euclidean space, two data points that are located on adjacent, overlapping layers of the "roll" might have a very small Euclidean distance, even though their true geodesic distance (the actual path along the unrolled surface) is quite large. A linear classifier would fail to cleanly slice through this curled geometry. Therefore, effective classification requires mapping the high-dimensional spatial coordinates into a lower-dimensional embedding that successfully unrolls the manifold while preserving local neighborhood structures.

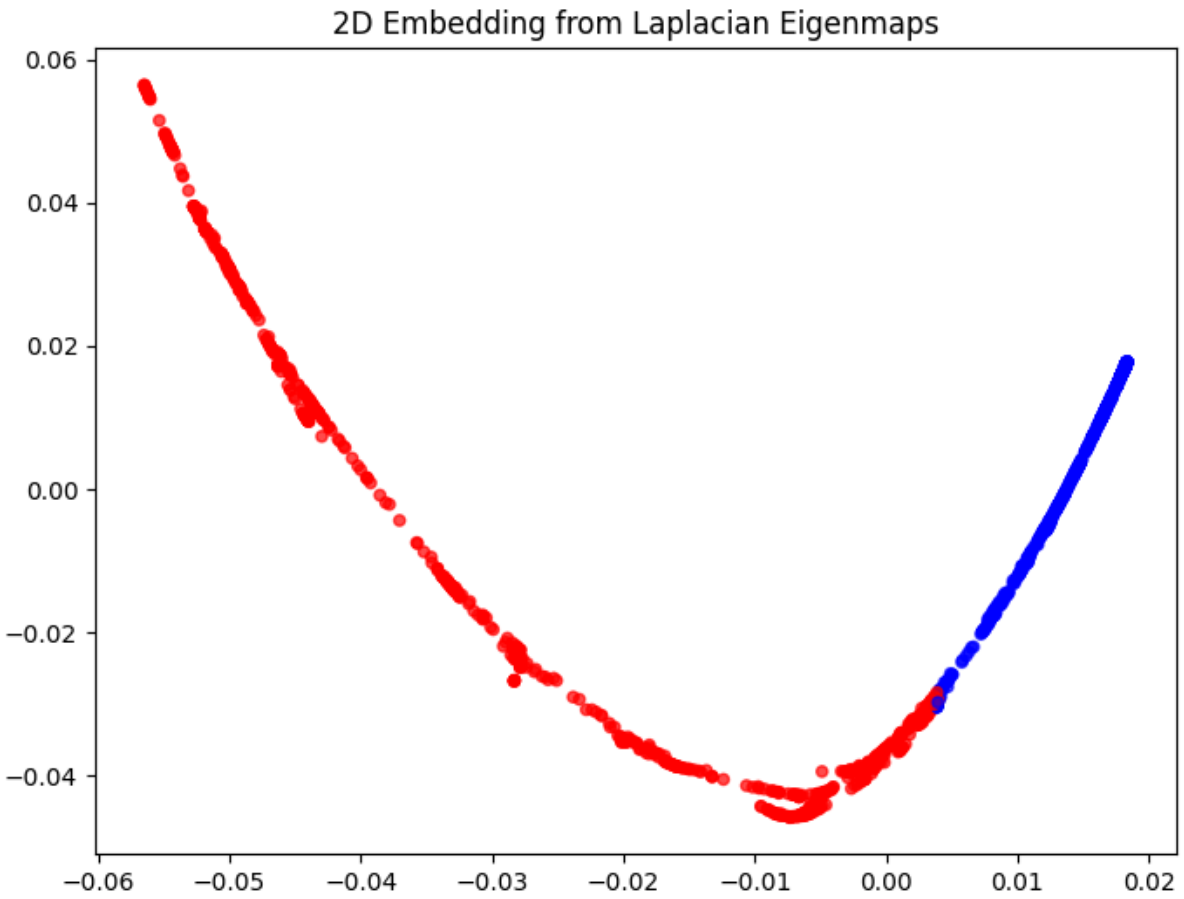


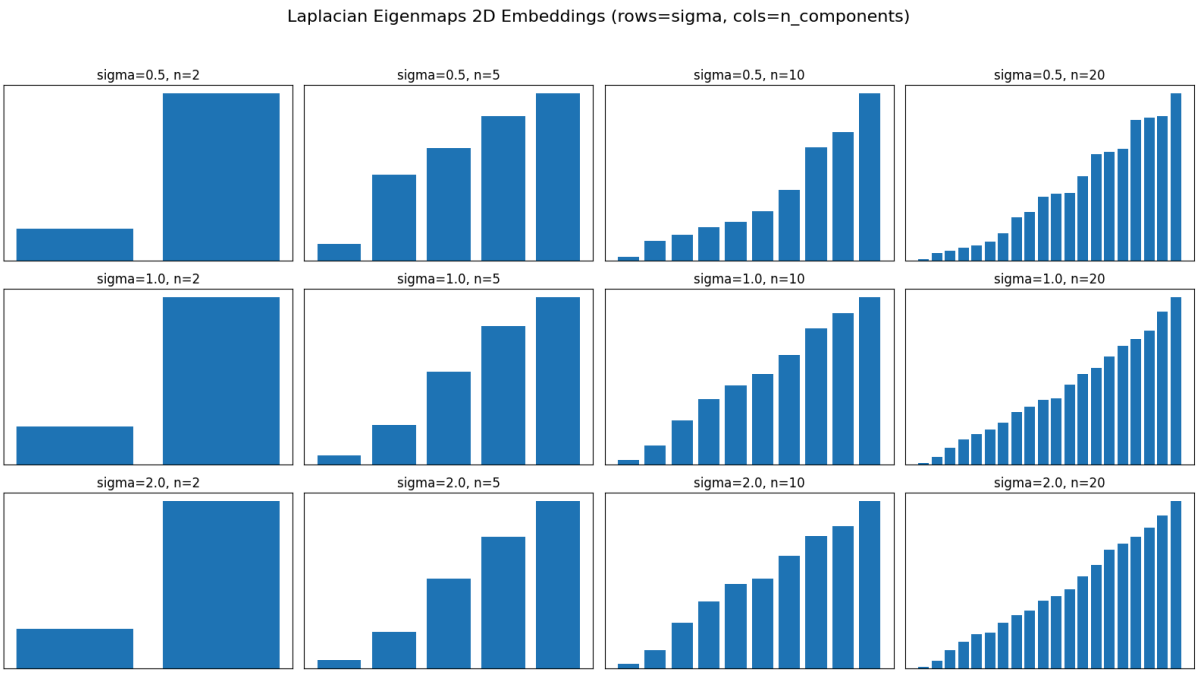


2. Laplacian Eigenmaps

Laplacian Eigenmaps were implemented entirely from scratch to build a graph representation of the data and compute the low-dimensional embedding using the graph Laplacian.

Methodology & Scree Plot: We evaluated the algorithm across various values of sigma ({0.5, 1, 2}) and dimensions k ({2, 5, 10, 20}). The eigenvalues were plotted in ascending order to create a scree plot. By analyzing the "eigengap"—the sharp jump between consecutive eigenvalues—we identified the optimal cutoff point for dimensionality. Eigenvalues before the gap represent dimensions that preserve the core geometry of the data with minimal distortion, while those after the gap introduce severe distortion by forcing closely connected points apart.


2D Embedding from Laplacian Eigenmaps
0.06
0.04
0.02
0.00
−0.02
−0.04
−0.06
−0.05
−0.04
−0.03
−0.02
−0.01
0.00
0.01
0.02


Laplacian Eigenmaps 2D Embeddings (rows=sigma, cols=n_components)
sigma=0.5, n=2
sigma=0.5, n=5
sigma=0.5, n=10
sigma=0.5, n=20
sigma=1.0, n=2
sigma=1.0, n=5
sigma=1.0, n=10
sigma=1.0, n=20
sigma=2.0, n=2
sigma=2.0, n=5
sigma=2.0, n=10
sigma=2.0, n=20

3. SNE and UMAP

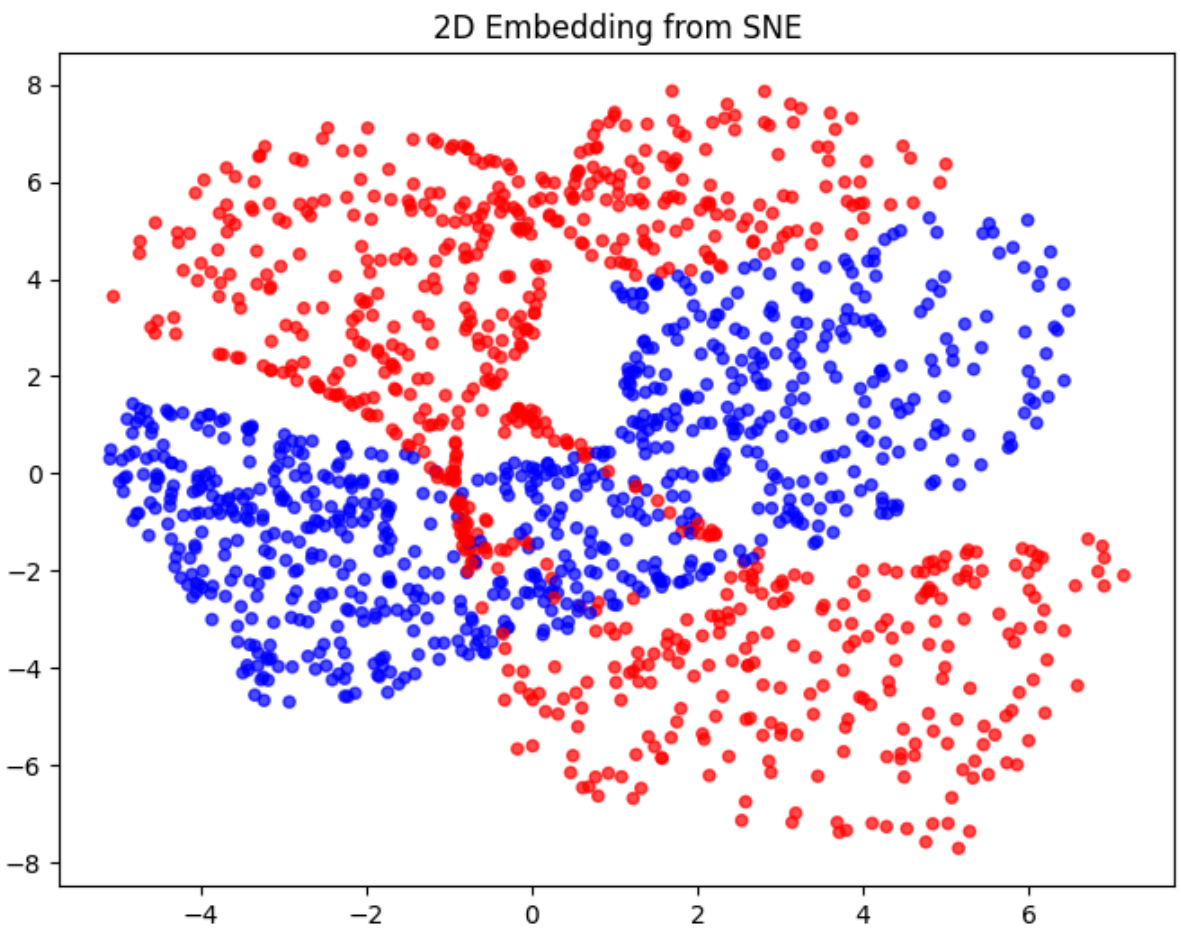
Both SNE and UMAP construct low-dimensional topological representations by modeling the probabilities of data points being neighbors.

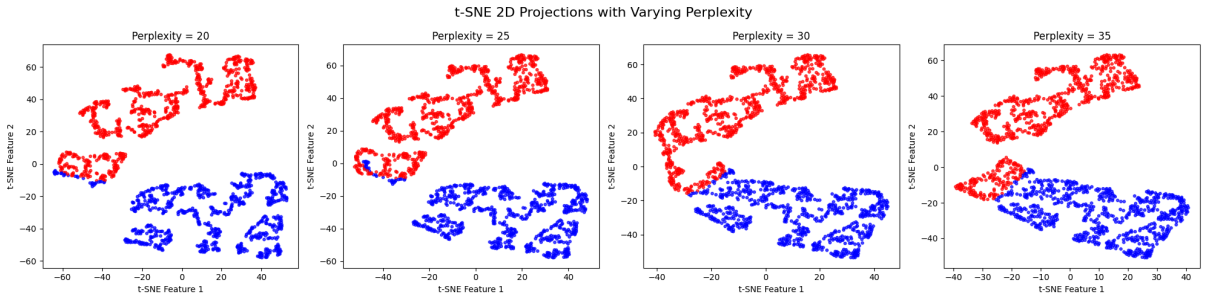
Parameter Tuning:

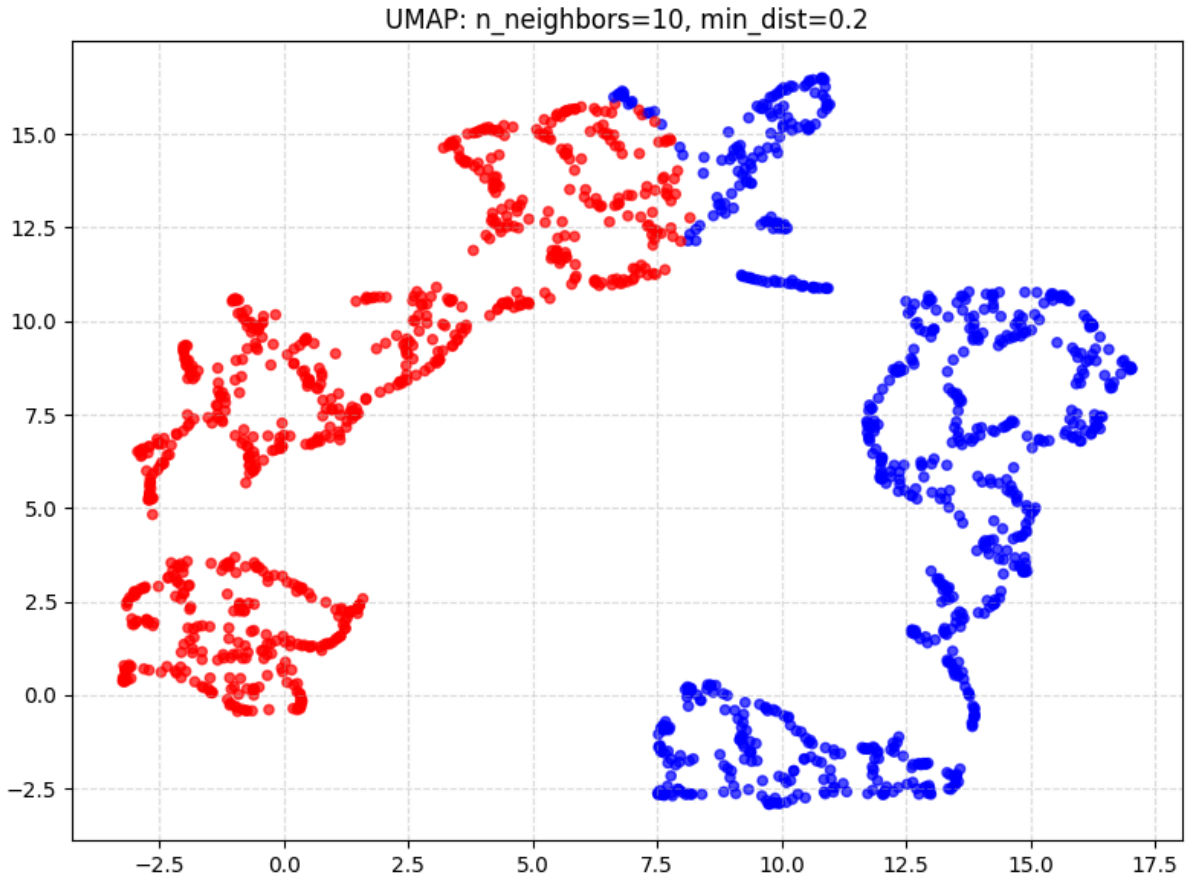
- **SNE:** Implemented using PyTorch. Instead of a dynamically calculated perplexity, a constant σ was utilized to represent the variance of the Gaussian distribution centered on each data point. A learning rate of 0.01 was used with an Adam optimizer. To ensure the model converged properly over the 2000 iterations without the learning rate vanishing prematurely, a ReduceLROnPlateau scheduler was utilized (monitoring the KL Divergence with a factor of 0.5 and a patience of 200 iterations).
- **t-SNE (Built-in):** The `sklearn.manifold.TSNE` implementation was utilized as a baseline. The critical parameter explored was `perplexity` (tested between 20 and 35), which dictates the number of nearest neighbors each point considers, balancing attention between local and global aspects of the dataset. The `n_iter` was set to 1000 to allow the algorithm sufficient time to resolve the complex Swiss Roll manifold.
- **UMAP:** For UMAP, the critical parameters are n_neighbors and min_dist. n_neighbors (set to 5-20 in testing) determines the size of the local neighborhood used for manifold approximation, effectively balancing local versus global structure. min_dist (set to 0.05) controls how tightly packed the embeddings are allowed to be, determining the aesthetic clustering of the 2D scatter plot.
- **UMAP (Built-in & Scratch):** For UMAP—evaluated using both the custom scratch implementation and the `umap-learn` library—the defining parameters are `n_neighbors` and `min_dist`. `n_neighbors` (tested 5 and 10) determines the size of the local neighborhood used for manifold approximation. `min_dist` (tested 0.05, 0.1, 0.2) controls how tightly packed the embeddings are allowed to be, which directly determines the visual density of the resulting 2D scatter plots. The built-in UMAP achieved results much faster due to underlying graph optimizations, though both successfully preserved local neighborhoods.

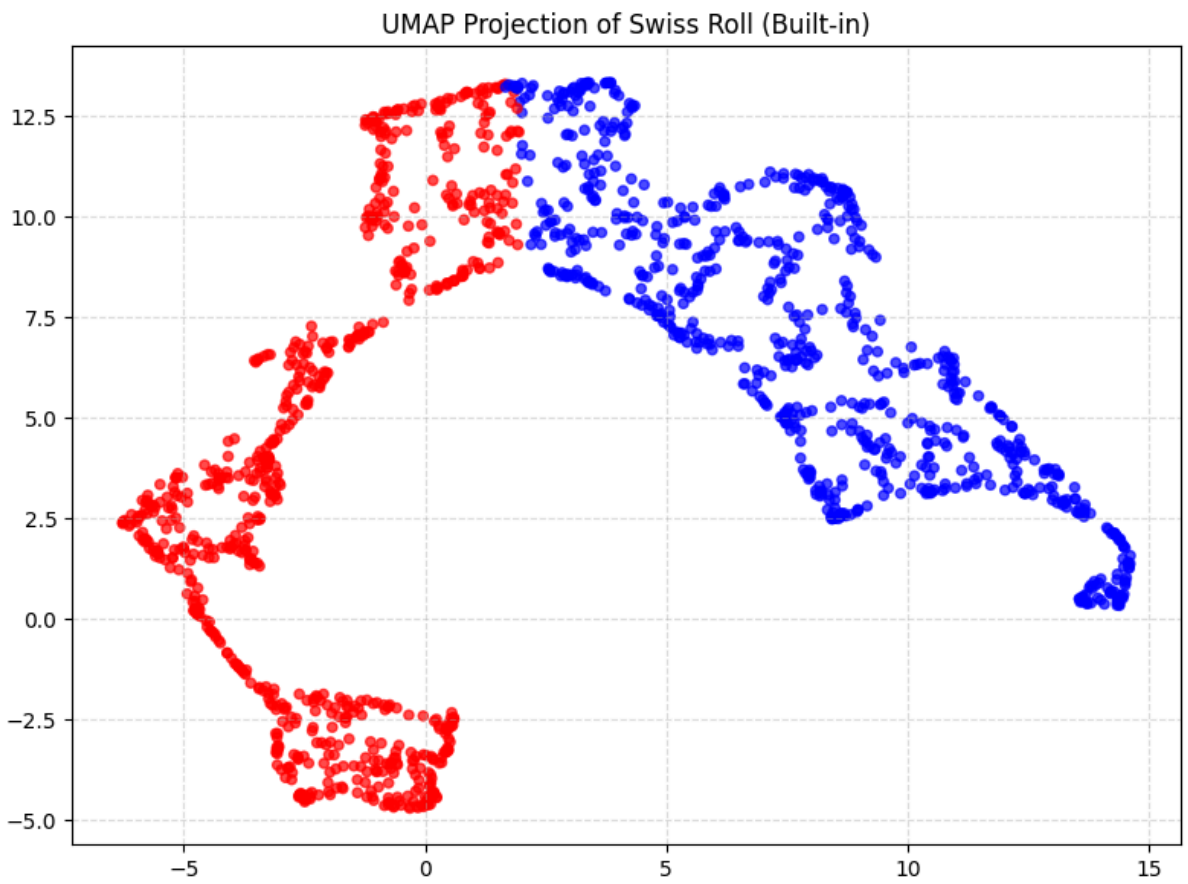
Code Snippet: SNE PyTorch Implementation (Gradient Descent Step)

```
for iter in range(n_iterations):
    optimizer.zero_grad()

    # Step 4: Compute pairwise distances in low-dimensional space
    sum_y = torch.sum(y**2, dim=1)
    dist_y = sum_y.view(-1, 1) + sum_y.view(1, -1) - 2 * torch.mm(y, y.T)

    # Step 5: Compute q_ij
    exponent = -dist_y
    # Mask out the diagonal so it doesn't affect the sum
    mask = torch.eye(y.shape[0], device=device)
    exponent = exponent - (mask * 1e9) # Effectively makes exp(-inf) = 0 on diagonal

    log_q = exponent - torch.logsumexp(exponent, dim=1, keepdim=True)
    # Step 6: Compute KL divergence and update embeddings
    kl_divergence = F.kl_div(log_q, p, reduction='batchmean')
    kl_divergence.backward()
    optimizer.step()
    scheduler.step(kl_divergence.item())

    if do_print and ((iter + 1) % 100 == 0):
        print(f"Iteration {iter + 1}/{n_iterations}, KL Divergence: {kl_divergence.item():.4f}")
```




t-SNE 2D Projections with Varying Perplexity
Perplexity = 20
Perplexity = 25
Perplexity = 30
Perplexity = 35
t-SNE Feature 1
t-SNE Feature 2


UMAP: n_neighbors=10, min_dist=0.2

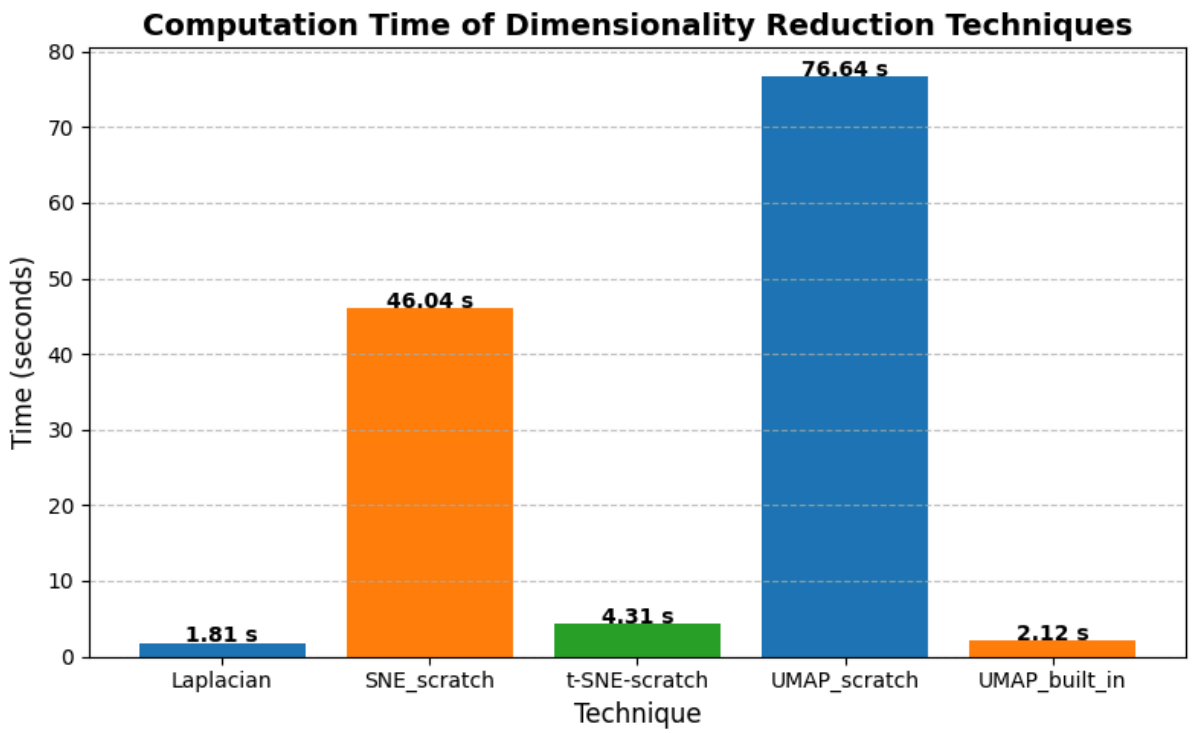


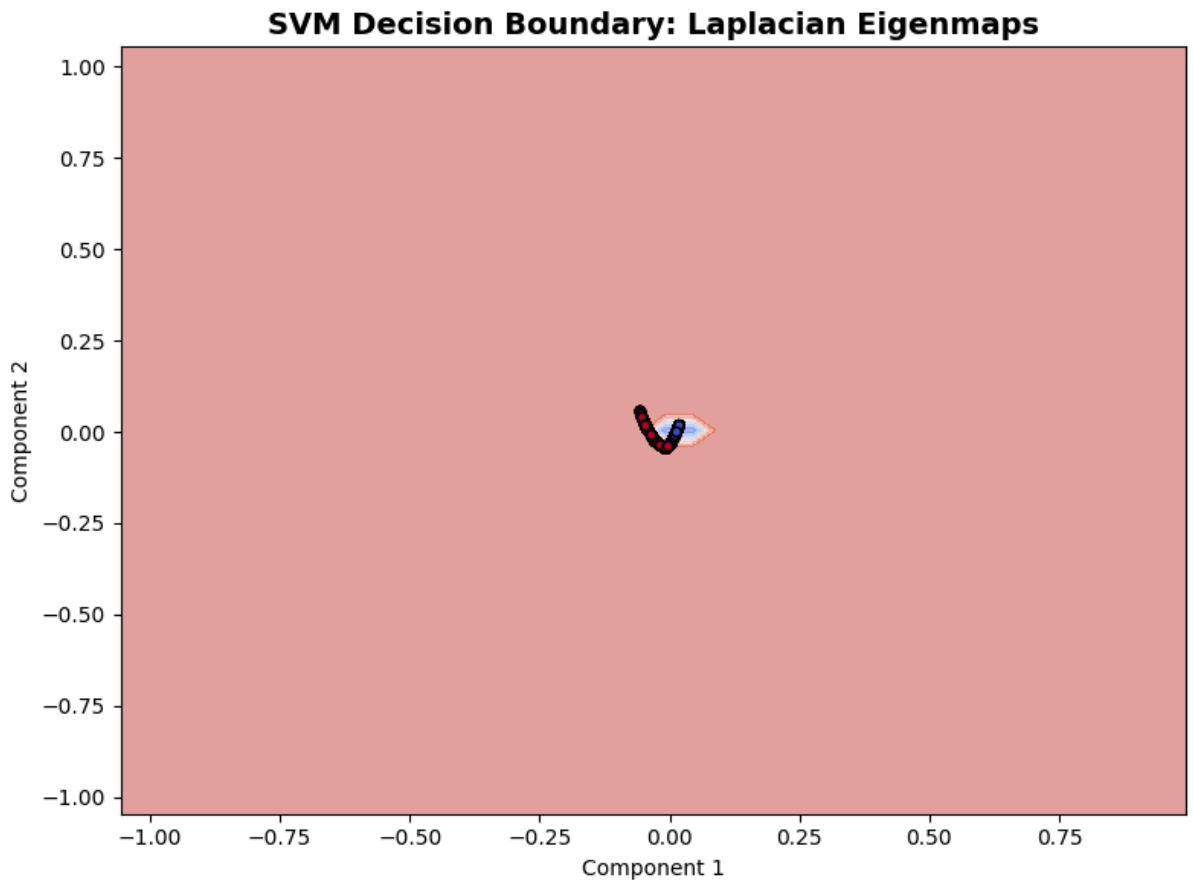
4. Classification with SVM

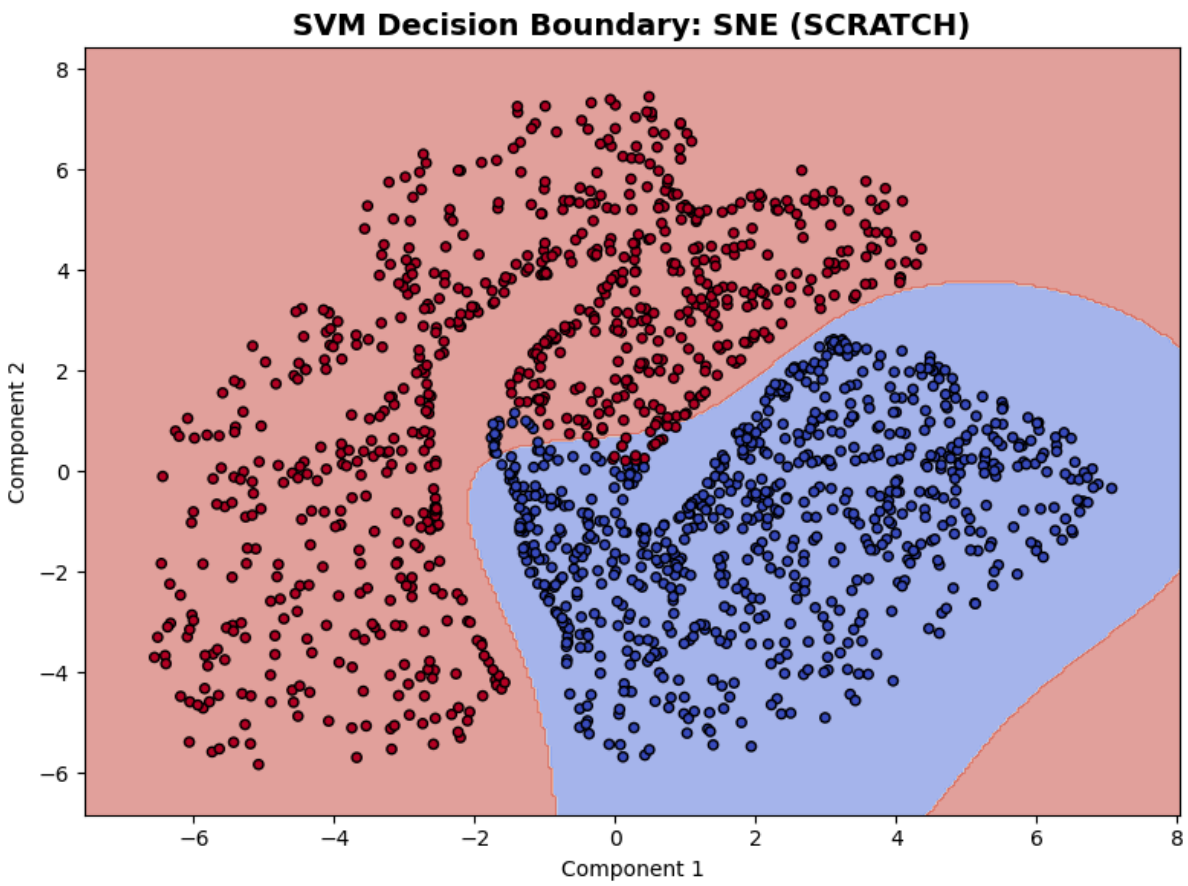
To measure the retention of classifiable structures, a Support Vector Machine (RBF kernel) was trained on both the original 3D dataset and the reduced 2D embeddings. To ensure rigorous evaluation, a Stratified 5-Fold Cross-Validation was used.

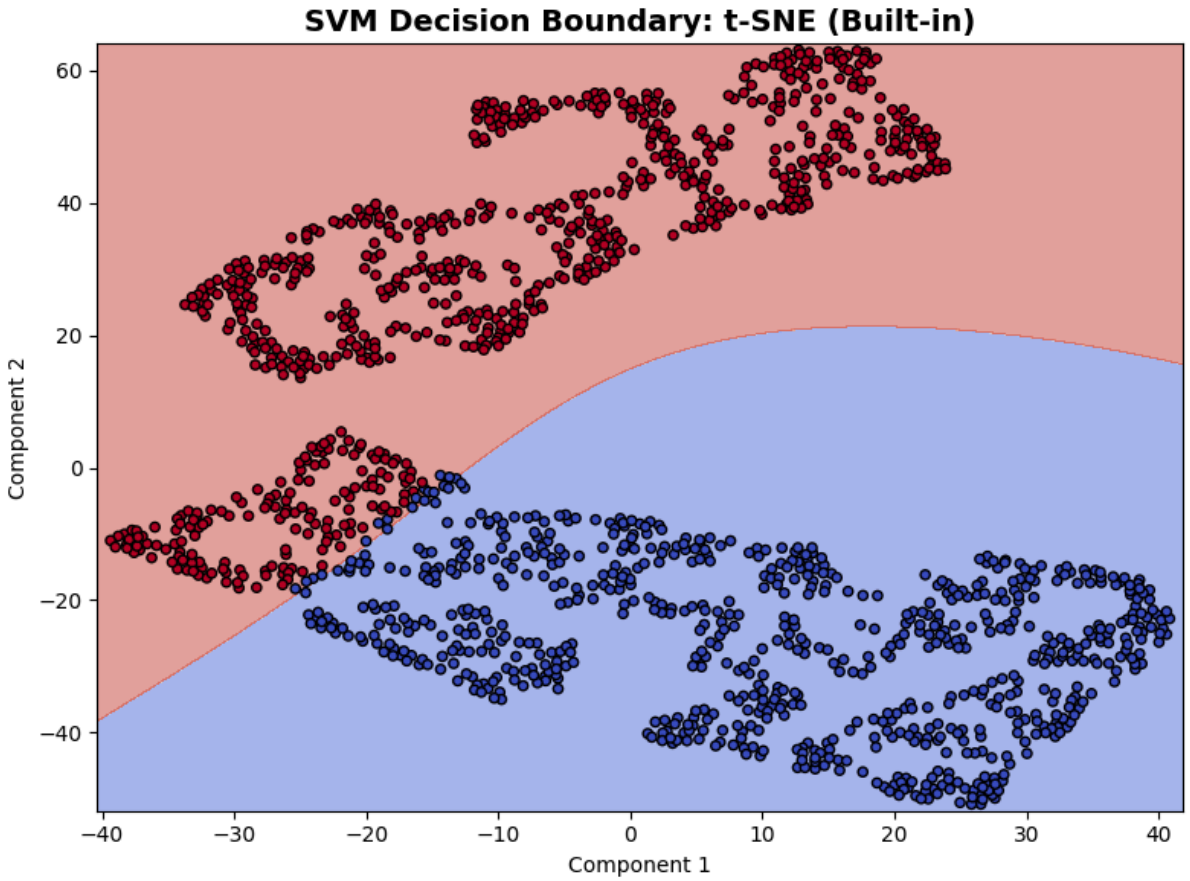
Results:

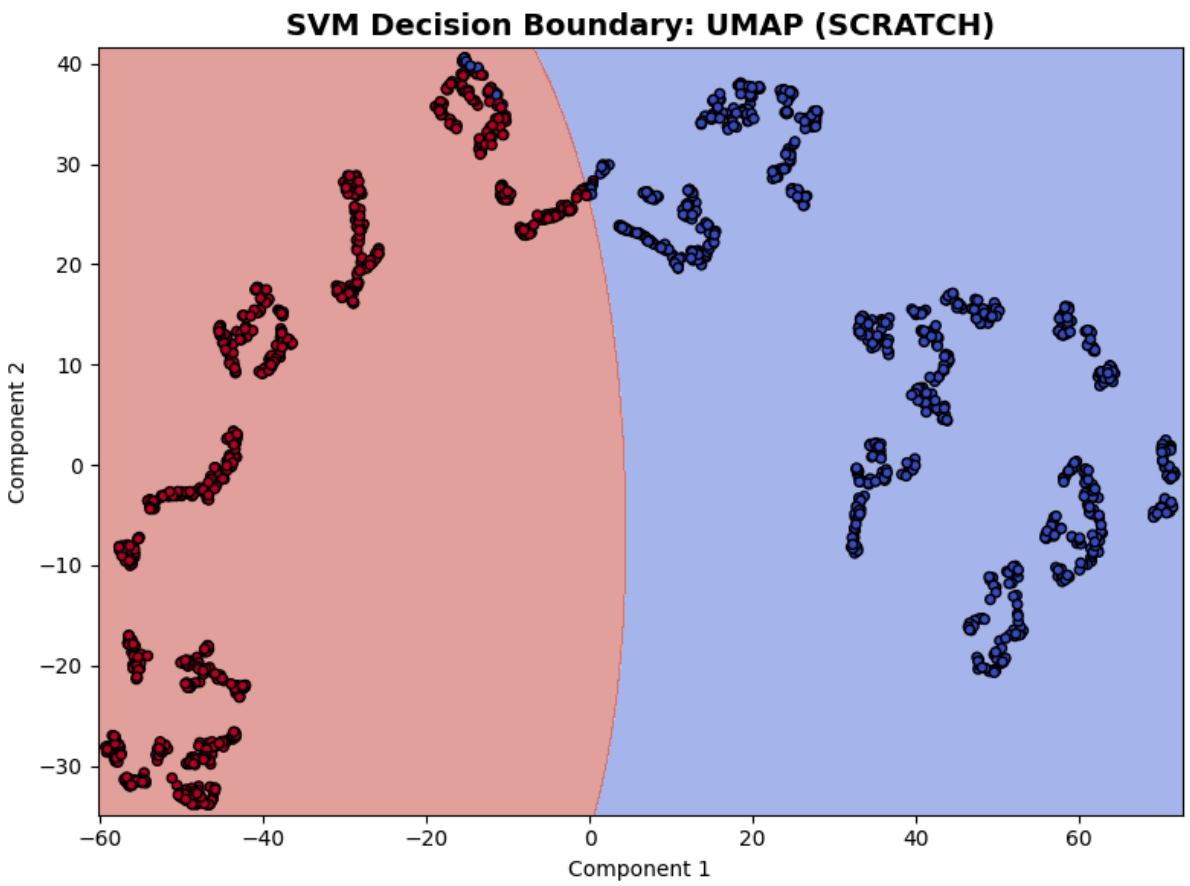
```
Training SVMs and calculating accuracies...
Running Stratified 5-Fold CV...
Original Data (3D): Accuracy = 99.73% (± 0.25%)
Laplacian Eigenmaps: Accuracy = 98.07% (± 0.53%)
SNE-Scratch: Accuracy = 97.73% (± 0.77%)
t-SNE-Built_in: Accuracy = 98.80% (± 0.34%)
UMAP-Scratch: Accuracy = 98.47% (± 0.27%)
UMAP-Built_in: Accuracy = 98.07% (± 1.02%)
```

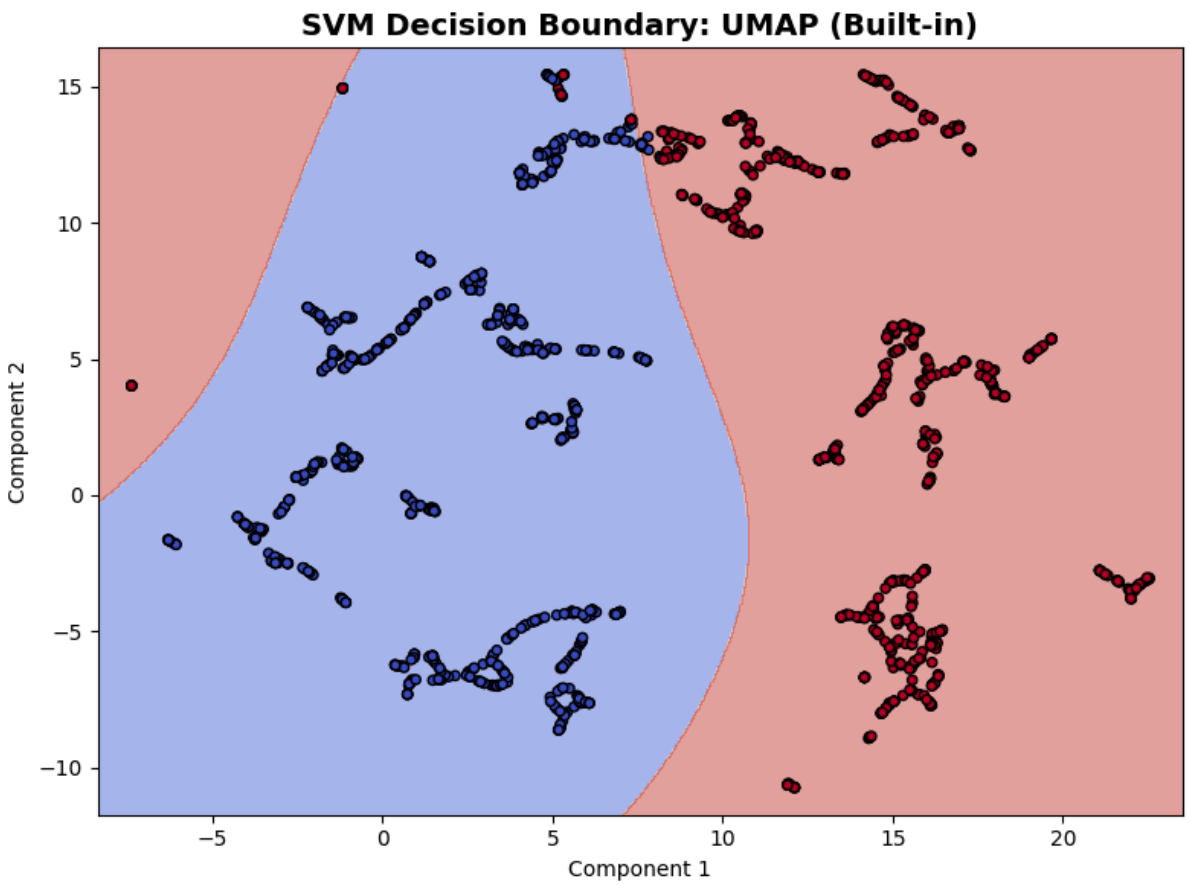

Computation Time of Dimensionality Reduction Techniques
Time (seconds)
Technique
1.81 s
46.04 s
4.31 s
76.64 s
2.12 s
Laplacian
SNE_scratch
t-SNE-scratch
UMAP_scratch
UMAP_built_in

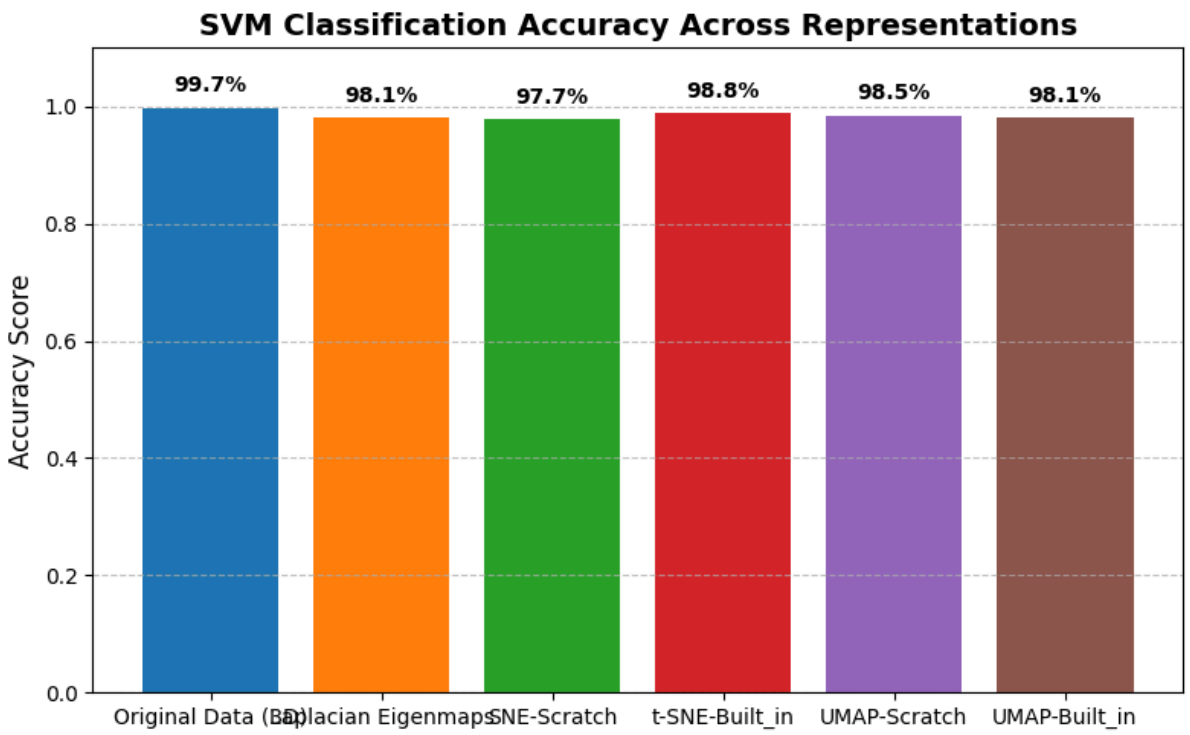

SVM Decision Boundary: Laplacian Eigenmaps
Component 1
Component 2


SVM Decision Boundary: SNE (SCRATCH)
Component 2
Component 1
8
6
4
2
0
−2
−4
−6
−6
−4
−2
0
2
4
6
8


SVM Decision Boundary: t-SNE (Built-in)
Component 2
Component 1
60
40
20
0
−20
−40
−40
−30
−20
−10
0
10
20
30
40


SVM Decision Boundary: UMAP (SCRATCH)
Component 2
Component 1
40
30
20
10
0
−10
−20
−30
−60
−40
−20
0
20
40
60


SVM Decision Boundary: UMAP (Built-in)
Component 2
Component 1
15
10
5
0
−5
−10
−5
0
5
10
15
20



5. Conclusions & Implications

The RBF SVM easily handles the non-linear separation in the original 3D space, yielding the baseline maximum accuracy of 99.73%. However, the dimensionality reduction techniques demonstrated immense utility by retaining nearly all of the structural integrity in just two dimensions.

Both built-in and scratch implementations yielded accuracies clustering tightly around 98%, confirming their effectiveness in preserving local neighborhood structures. Overall, the results confirm that successfully unrolling the underlying non-linear manifold facilitates robust, low-dimensional classification with minimal loss of critical data topology.

.